2026 전국 대학생 AI 반도체 회로 설계 경진대회 · Digital 트랙 2차 결과

회전하는 이벤트 카메라를 하나의 월드 지도로 옮기는 저전력 회로

Event camera world-coordinate transform & world memory (RTL, P&R)

팀 REDRED

문제 정의: 센서가 돌아도 지도는 제자리에 있어야 한다

입력

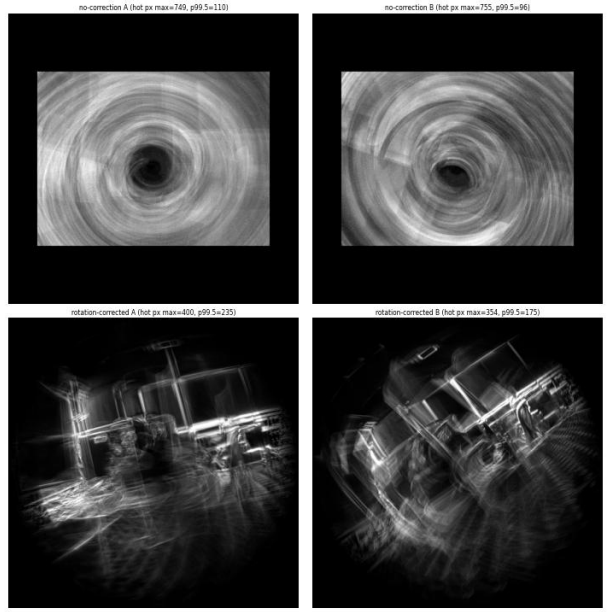
센서 960×720
이벤트 = 주소 (x, y) + 극성
회전각 θ (외부 공급 또는 추정)

제약 (교수 Q&A 확정)

회전 180° 이내
평행 이동 없음 (미세하게만)
표면만 보는 2차원 월드 메모리

출력

월드 메모리
칸마다 이벤트 유무 + 극성
호스트용 읽기 포트



회전을 보정하지 않고 쌓으면(위) 소용이로 번지고, 회전각으로 좌표를 바꿔 쌓으면(아래) 방의 구조가 보인다. (실제 DELTA 센서 영상, 소프트웨어 모델)

우리가 정한 평가 관점

- 이벤트를 잃지 않고 정확한 칸에 쓰는가
- 구현 가능한 면적·전력인가
- 회로가 실제 데이터에서 안정적인가
- 한계를 스스로 설명할 수 있는가

타겟은 로봇팔 손목 카메라, 크기는 이유와 함께 제안한다

항목	RTL로 검증한 크기	로봇팔 목표(제안)	근거
센서 패치	4× 4	256× 256	30~ 50 cm 근접 작업에서 부품 디테일 인식 필요
월드 메모리	1024× 1024 (좌표 변환·추정기) 64× 64 (쓰기 경로+ 메모리 통합)	1024× 1024	회전 180° 이내라 파노라마처럼 무한히 커지지 않음, 패치의 2~ 4배면 충분
회전각 단계	1024 (좌표 변환·추정기) 256 (통합)	1024	약 0.18°/단계(180°/1024), 그립 정밀도

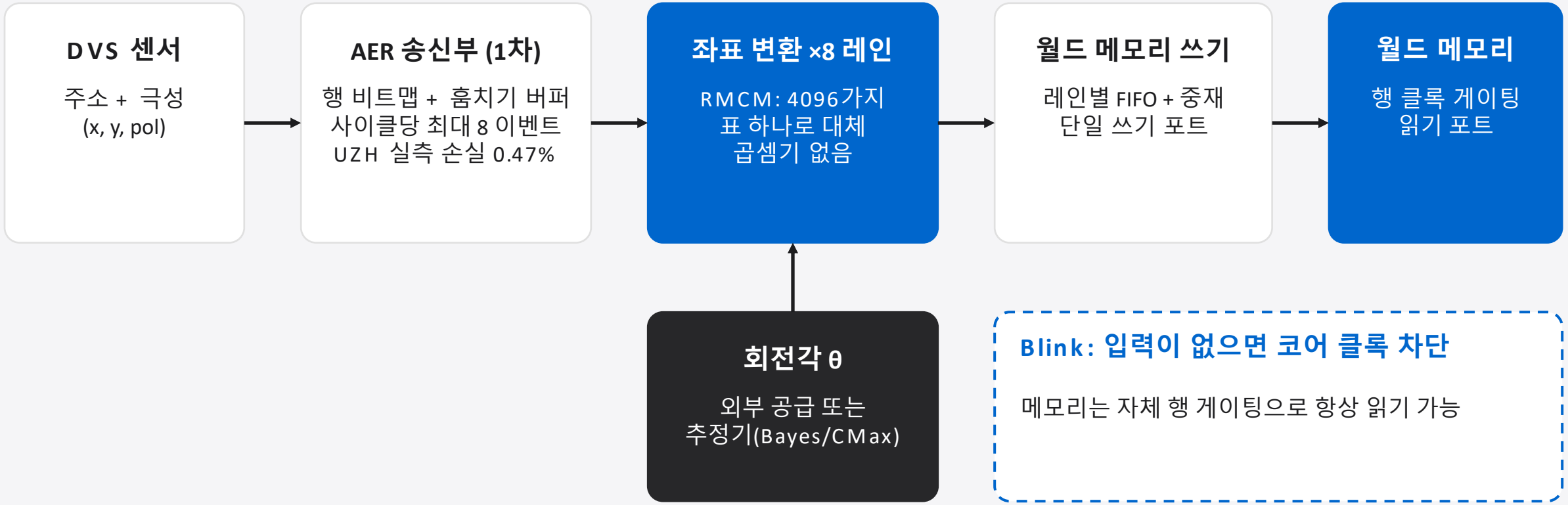
왜 로봇팔인가

회전만 하고 이동이 없다는 전제와 국소 스캔 동작이 일치한다. 센서 스펙(960×720, 6 mm 렌즈)도 3차 Q&A로 확정됐다.

제외한 시나리오

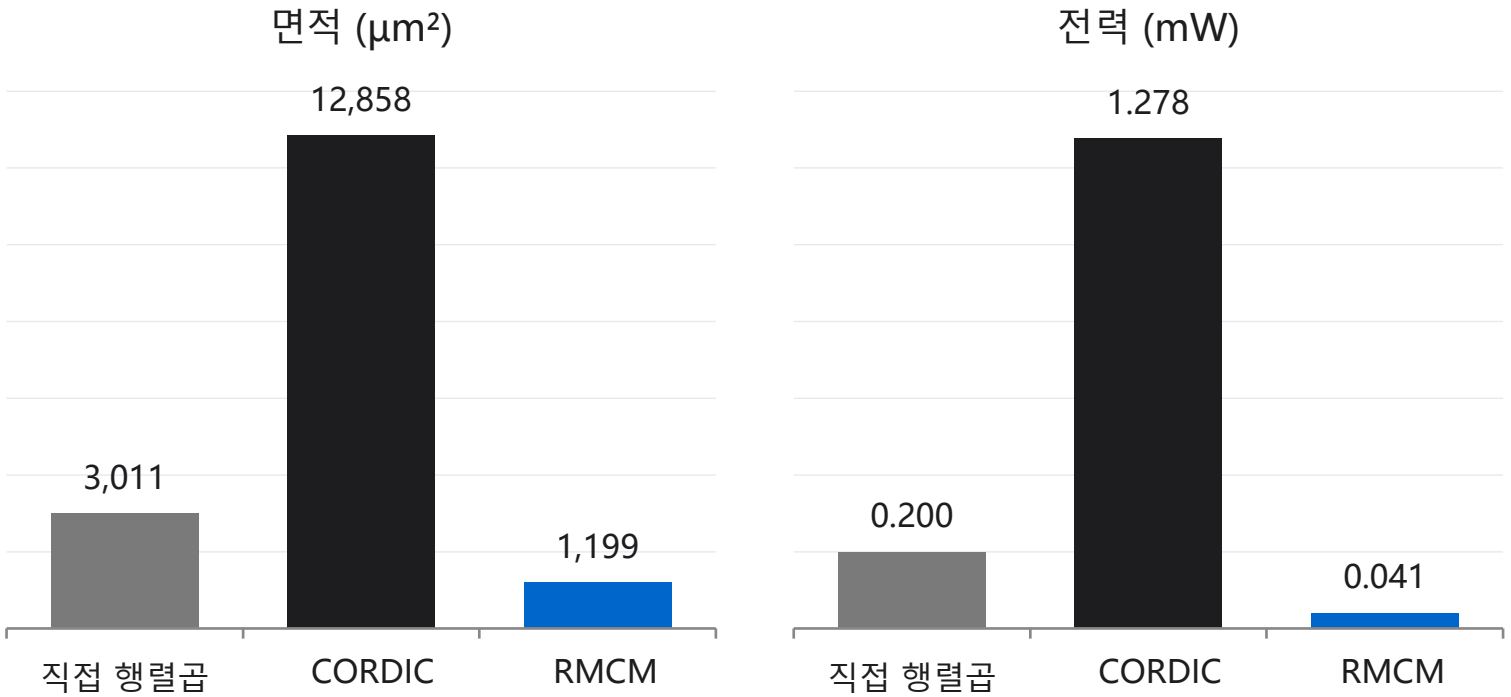
자율주행: 이동이 지배적이라 전제가 깨짐
PTZ·산업 스캐너: 해상도 확장이 가장 어려워 후순위

구조: 1차 송신부 위에 좌표 변환과 월드 메모리를 얹었다



핵심 선택 3가지: ①계산 대신 표 ②쓰기 포트 하나로 직렬화 ③메모리를 지배적 비용으로 보고 클록을 끄는 구조

좌표 변환은 계산을 줄이지 않고 아예 표로 바꿨다



200 MHz 타이밍 여유	슬랙
직접 행렬곱	+1 ps (턱걸이)
CORDIC	−252 ps (위반)
RMCM	**+2,187 ps**

Genus 합성, 5 ns, GPDK045. 입력 4096가지 (4×4 로컬 좌표 × 256각도) 기준. RMCM은 직접 행렬곱과 bit-exact로 같은 값을 냄(근사 오차 0).

곱셈기를 없애는 것이 항상 이득은 아니다. 입력 후보가 몇 가지로 좁을 때는 계산을 없애고 표로 대체하는 쪽이 이겼다. (다. (이 결론은 입력 공간이 큰 경우엔 달라질 수 있다.)

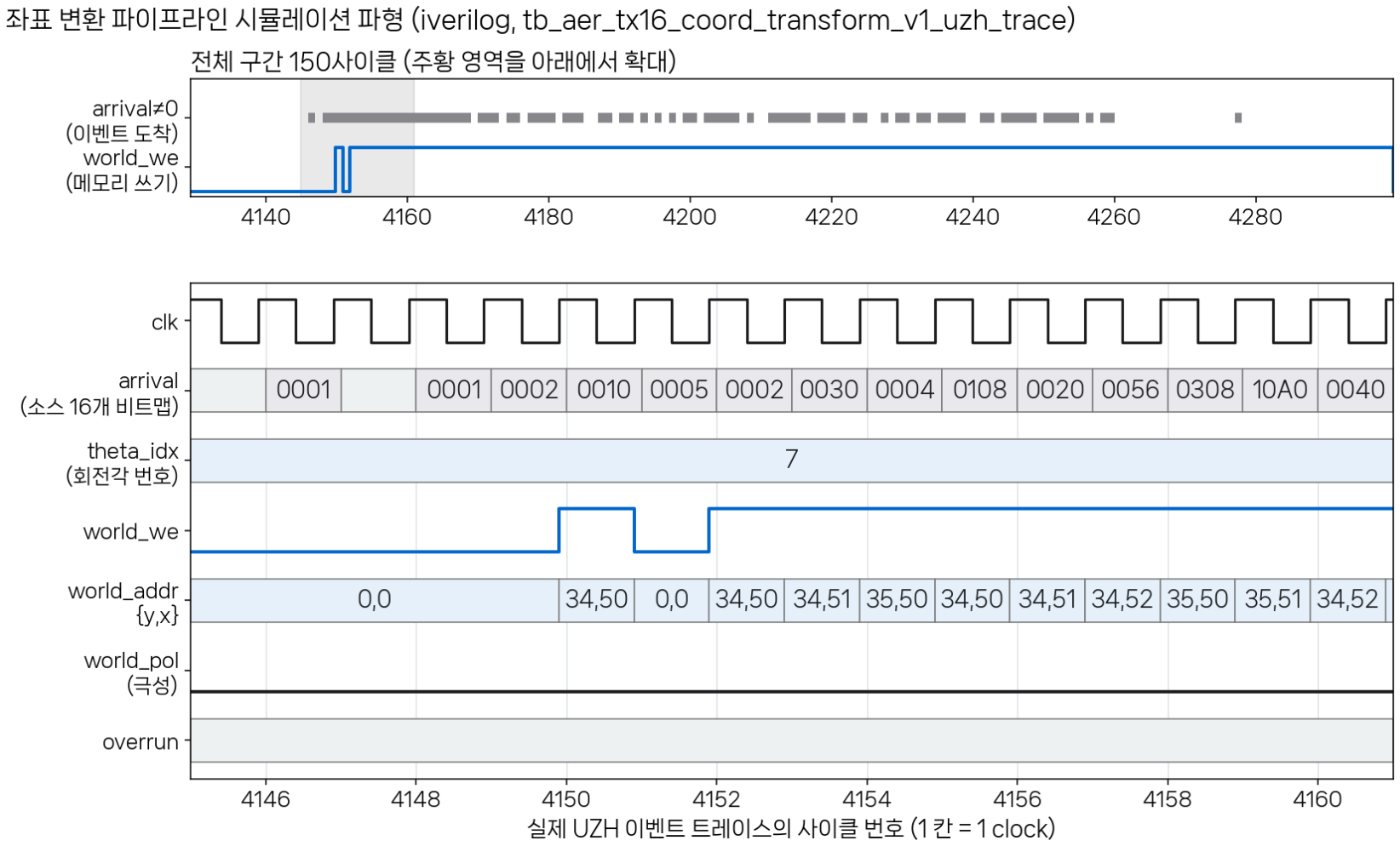
검증: 소프트웨어 기준값과 비트 단위로 모두 일치했다

검증 항목	범위	결과
좌표 변환 전수 검사 (RMCM RTL)	16 로컬 좌표 × 1024각도 = 16,384가지	**16,384/16,384 일치**
실제 이벤트 재생 (UZH, 통합 회로)	8,503개 이벤트 + 실제 자세	**불일치 0, 손실 0, 90칸 기록**
무작위 스트레스	20,000 사이클, 20% 도착률, 매 사이클 θ 변경	**불일치 0 (20,241건), 528칸 기록**
θ 추정기 RTL vs 오라클	UZH 이벤트 300개의 매 추정값 (1024각도, 외부 SRAM 포트 버전)	**불일치 0**
blink vs 항상-켬 코어	유휴 100~6000 사이클	**overrun•메모리 이미지 불일치 0**
게이트 시뮬레이션 (P&R 후 netlist + signoff SDF)	무작위 6,000사이클 + 메모리 4,096칸 읽기	**RTL과 출력 10,097줄 일치**

검증이 잡아낸 실제 버그

① 등록형(registered) 파이프라인에서 θ가 한 박자 늦게 잡히는 것을 몰라 기대값이 어긋남 (테스트벤치, 2회)
② 추정기 RTL: 같은 사이클에 레지스터를 갱신하고 그 값으로 분기해 갱신 전 값을 읽는 버그 → 최종값을 조합논리로 먼저 계산하도록 수정

테스트벤치 파형: 이벤트가 레인을 거쳐 월드 주소로 나온다

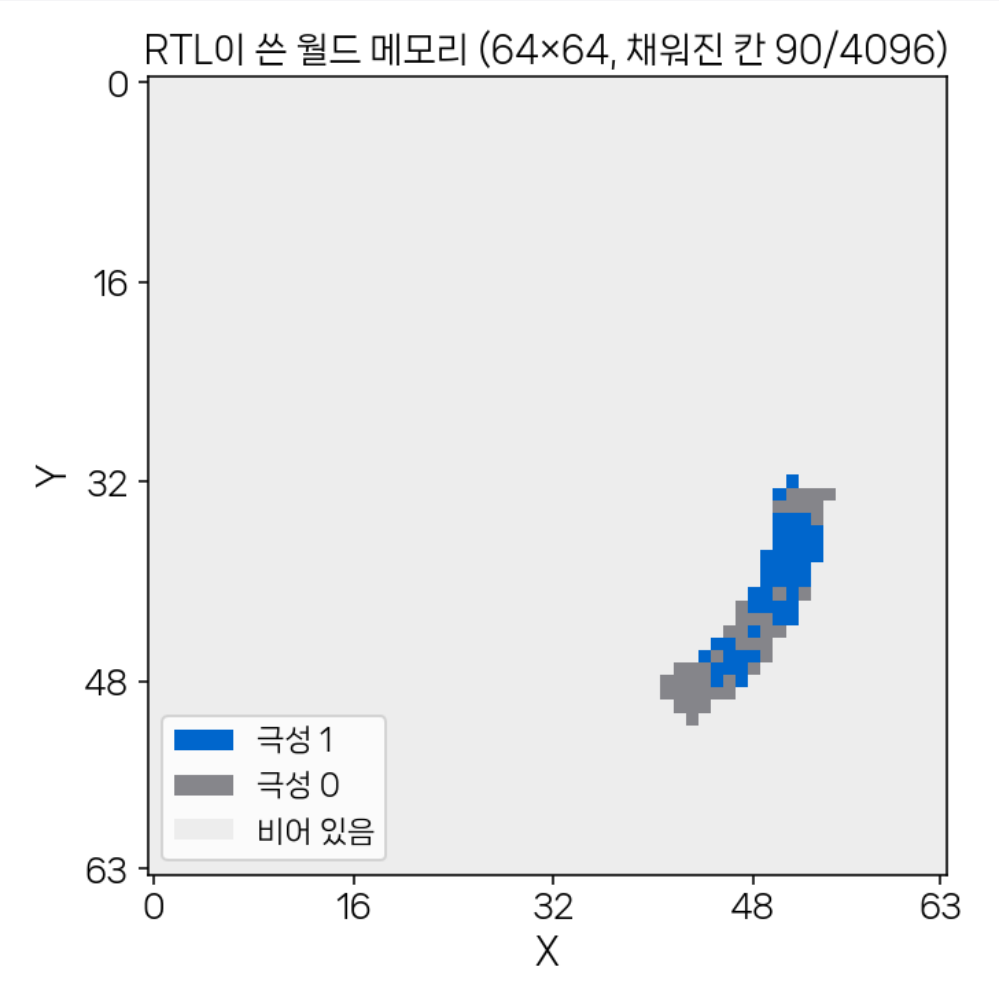


읽는 법

- arrival: 16개 소스 중 어디서 이벤트가 왔는가(16비트 비트맵)
- theta_idx: 그 순간의 회전각 번호
- world_we · world_addr: 변환된 월드 좌표(y,x)가 쓰기 포트로 한 건씩 나옴
- overrun은 0으로 유지

RTL 시뮬레이션(Icarus Verilog), 실제 UZH 이벤트 + 실제 자세

RTL이 쓴 월드 메모리: 8,503개 이벤트가 90칸에 쌓였다



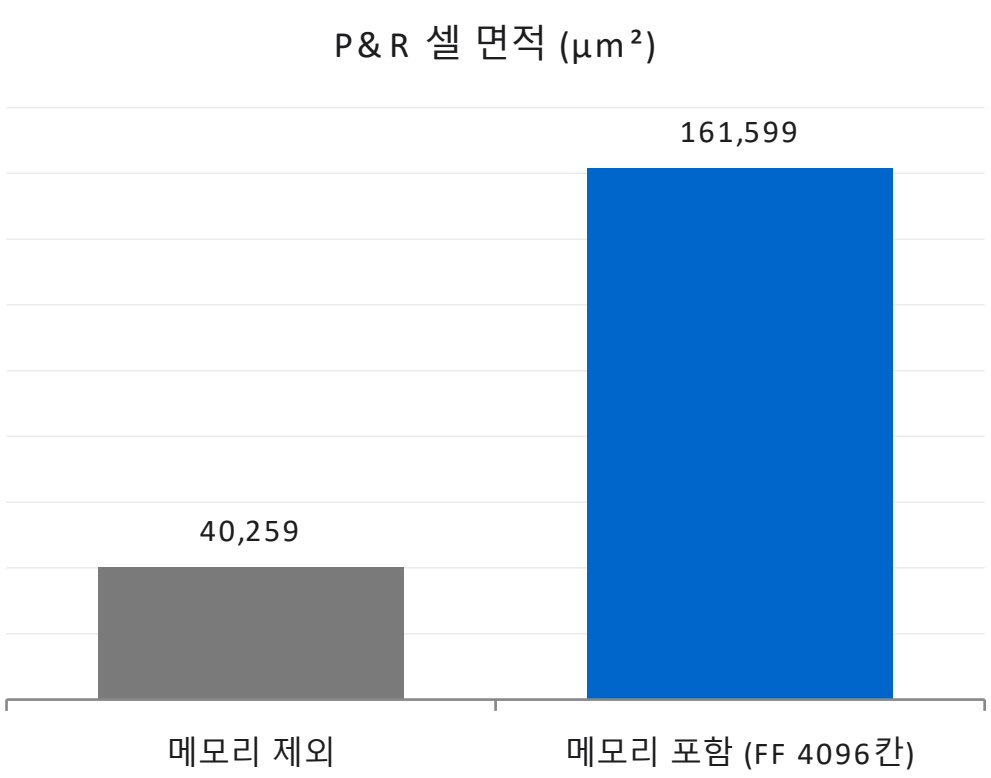
UZH 실제 이벤트 재생

이벤트 8,503개 → 쓰기 8,503건, 손실(overrun) 0
쓰기 모듈 입력 8,503건을 독립 계산한 기대값과 대조: 불일치 0
기록된 칸 90/4096

무작위 스트레스

20,000 사이클, 도착률 20%, 매 사이클 θ 변경
쓰기 20,241건 모두 기대값과 일치, 손실 0
기록된 칸 528/4096 (희소한 실제 트래픽이 놓치는 같은 사이클 충돌 돌 경로를 자극)

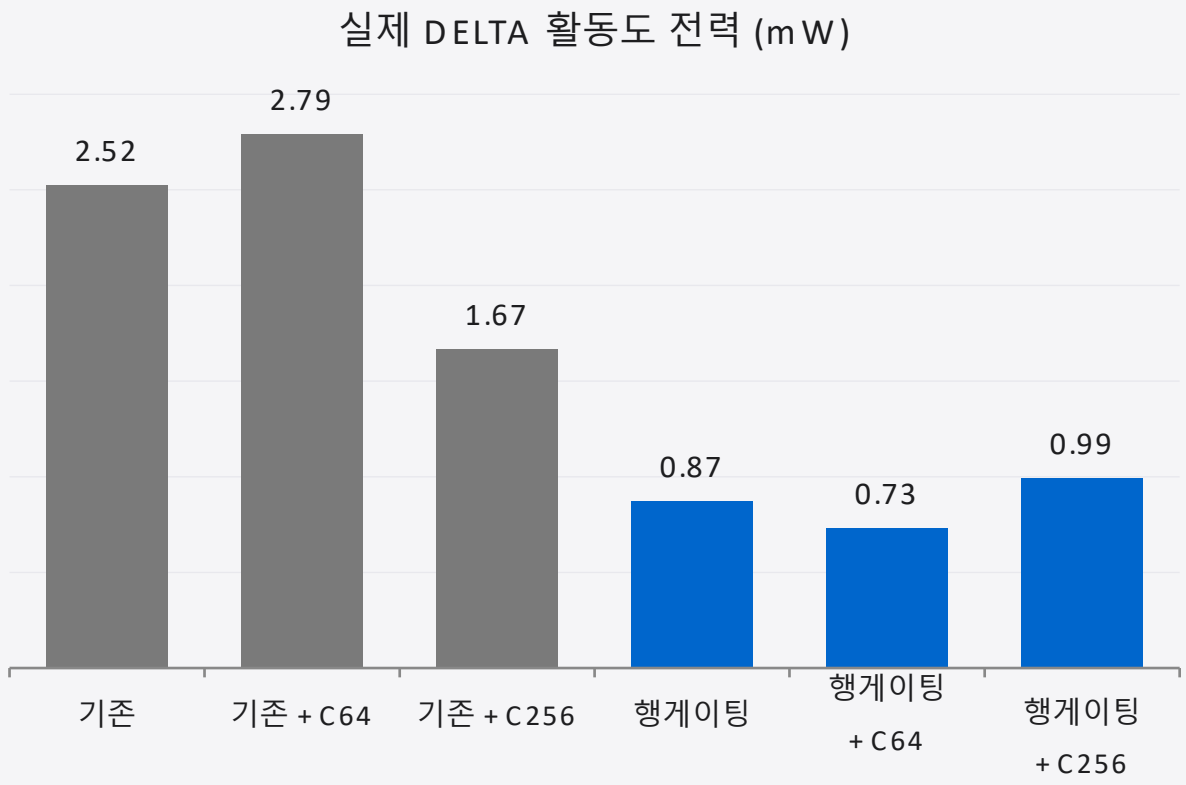
면적의 70%는 웰드 메모리, 그래서 메모리를 먼저 다뤘다

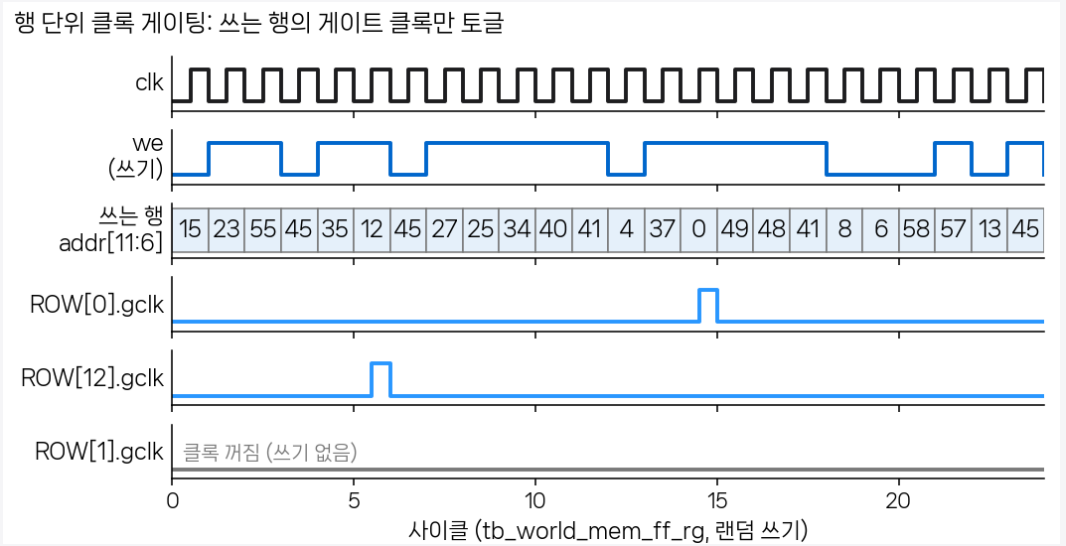


- 쓰기 모듈 + 레지스터 배열 합성: 211,856 µm², 전체의 98%가 저장소
- 저장소를 SRAM 인터페이스 밖으로 분리: 35,111 µm² (6.0배 감소). 단, 저장소는 별도 계산 대상
- 메모리 포함 P&R: 메모리가 면적의 70% 이상, 약 12 µm²/비트
- 교육용 PDK에는 SRAM 매크로가 없다 → 메모리는 별도 추정 모델이 필요

→ 이후 최적화의 초점은
코어 로직이 아니라
메모리의 클록 동작에 맞췄다.

행 단위 클록 게이팅으로 메모리 전력을 65% 줄였다





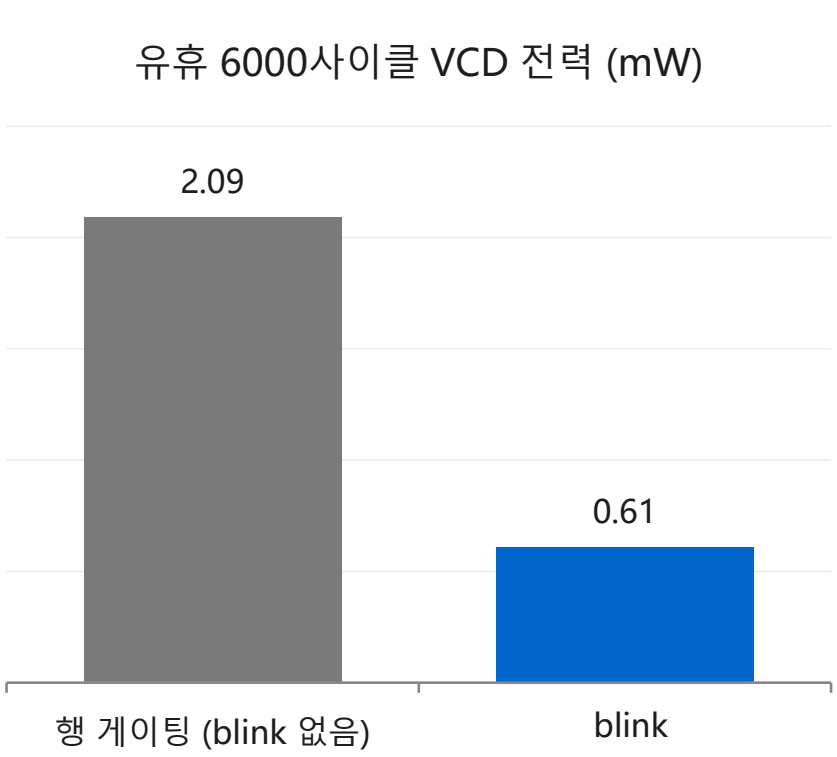
RTL 시뮬레이션 파형: 쓰는 행의 게이트 클록만 토글

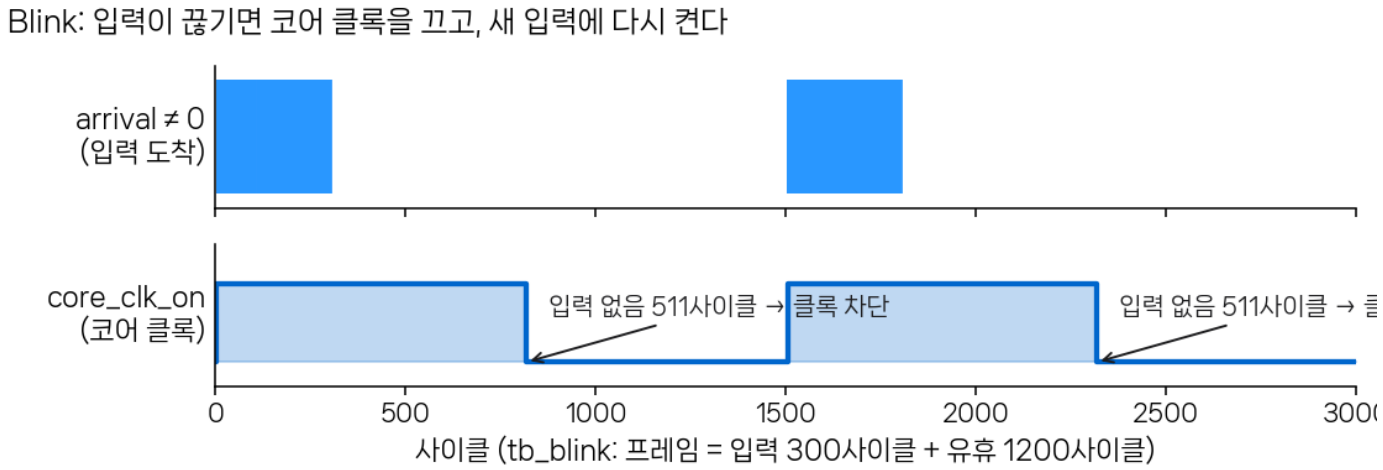
결과

전력 2.522 → 0.870 mW (−65.5%)
면적 90,105 → 85,617 µm² (− 5.0%)

C64·C256 = 쓰기 캐시 64·256항목. 원리: 행(y)마다 클록 게이트를 두어 쓰는 행의 128개 FF만 클록을 받는다(리셋 때만 전 행). 쓰기 캐시(64항목)는 전 − 16%, 면적 + 5.5%로 이득이 작아 필수로 두지 않았다. 256항목은 오히려 전력이 늘었다.

Blink: 입력이 없는 동안 코어 클록을 꺼 유휴 전력을 줄였다





RTL 시뮬레이션 파형: 입력이 끊기면(511사이클) 코어 클록 차단, 새 입력에 재가동

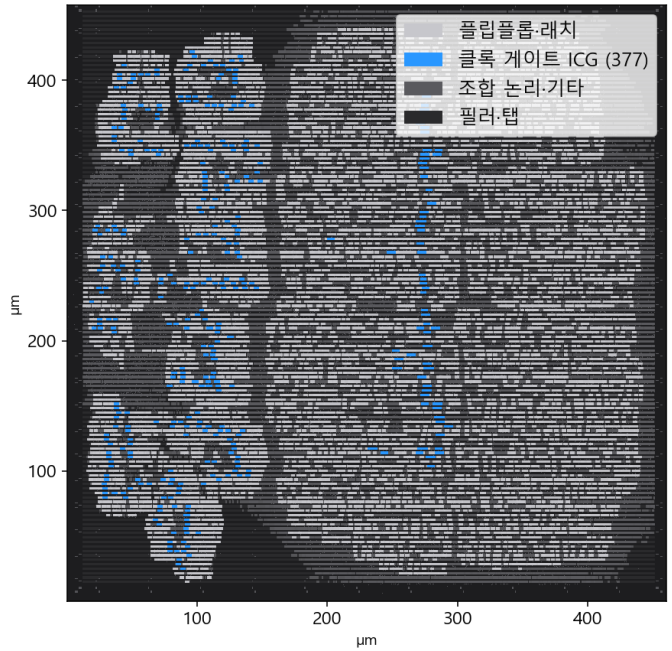
200 MHz에서 칩이 일하는 비율 (실제 DELTA)	비율
입력 8개/사이클 버스트 가정 (하한)	3.1% (p90 5.4%)
쓰기 포트 1건/사이클 상한 가정	25%

주의: –71%는 유휴가 길도록 만든 시험 입력의 값이다. 실제 부하에서의 절감은 위 가동 비율에 비례하며, vectorless 전력에서는 두 구성이 같게 나온다(활동
다(활동도를 모르면 코어가 계속 켜져 있다고 가정).

물리 검증: 클록 게이팅 hold는 구조로 해결, 일반 경로에 ps 단위 잔여

항목 (signoff)	래치+AND 분리	통합 ICG	통합 ICG @5.2 ns
DRC / antenna	0 / 0	0 / 0	(진행 중)
클록 게이팅 hold	−0.030 ns (13)	+0.080 ns (0)	(진행 중)
reg2reg hold	−0.010 ns (135)	−0.007 ns (34)	(진행 중)
setup	위반 0	−0.019 ns (1)	(진행 중)
전력 (vectorless)	5.89 mW	6.11 mW	(진행 중)
표준셀 면적	159,459 µm²	162,312 µm²	(진행 중)

괄호 안은 위반 경로 수. 5 ns, util 0.6, GPDK045, Quantus 추출.



P&R 셀 배치도 (플립플롭 = 연회색, 클록 게이트 = 파랑)

원인

CTS가 래치를 말단 싱크로 균형 맞춰 AND 이트보다 약 0.2 ns 일찍 열림

수정과 시도

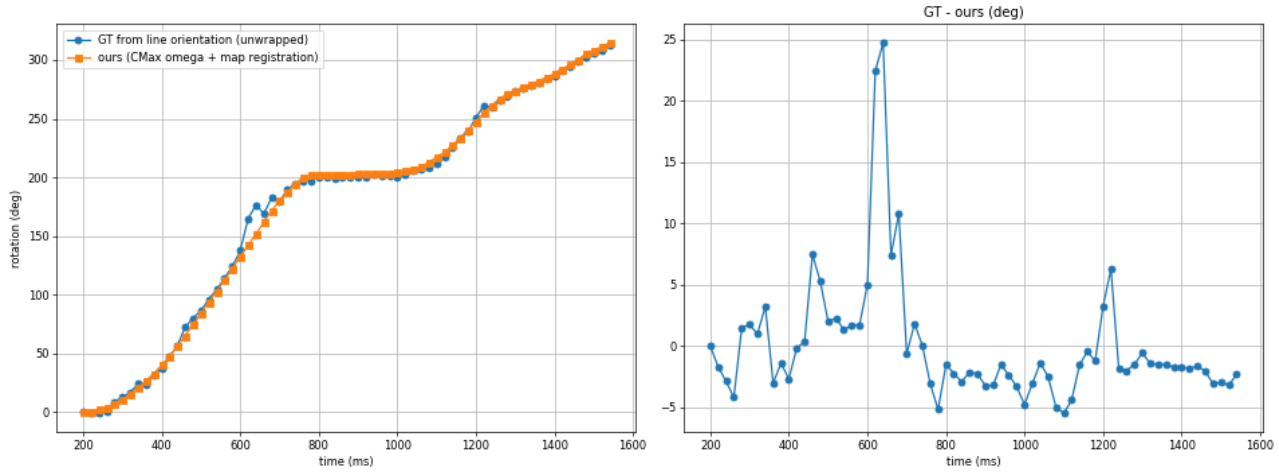
래치+AND → 통합 클록게이트 셀로 hold 해결. signoff ECO는 setup −0.040 ns로 악화되어 미채택

남은 것

일반 경로 ps 단위 잔여 위반. 클록 여유로 흡흡수하는 것이 현재 해법

회전각은 인과적으로만 추정한다: 미래가 필요한 방법은 버렸다

방법	실시간 가능	UZH 20곳 평균 / 최악 오차
Contrast maximization (순수 인과)	가능	11.7° / 59.4°
+ forward-backward 평활	불가 (미래 필요)	10.2° / 44.3°
batch bundle adjustment	불가 (미래 필요)	11.5° / 28.7°
bounded-lag (L=8 윈도우, 128 이벤트 FIFO)	가능 (지연 L)	10.6° / 50.3°



실제 DELTA 영상: 장면 속 직선 방향으로 만든 회전각 정답(파랑) vs 우리 추정(주황), 오른쪽은 오차(°)
차(°)

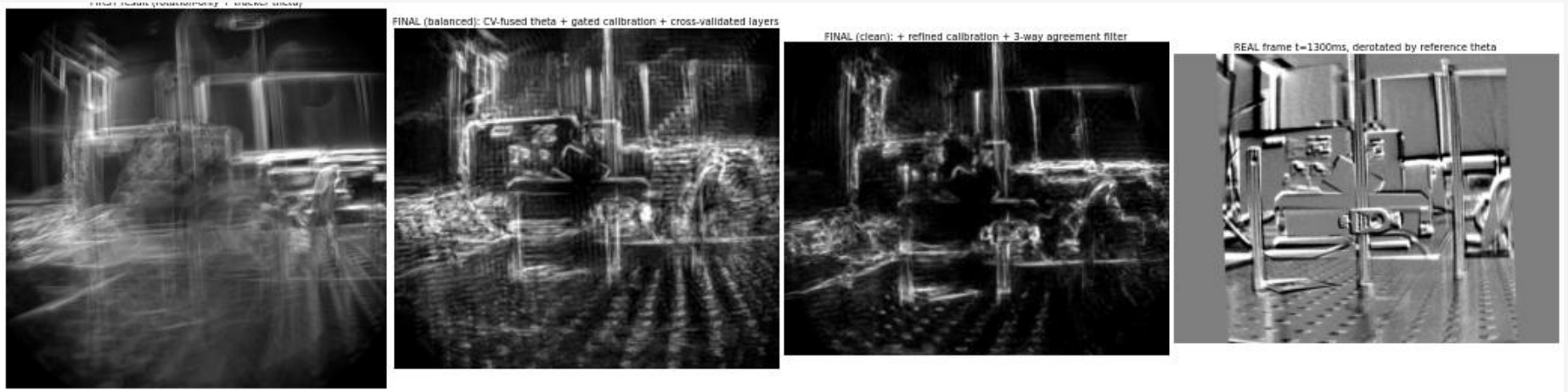
추정기 RTL (Bayes filter)

이산 베이즈 필터를 고정소수점으로 구현. UZH 이벤트 300개
개 재생에서 오라클과 비트 단위로 일치(1024각도 버전).

정직한 한계

정확도는 장면에 크게 의존한다. 로봇팔 실측(RobotEvt)에서
서는 UZH용 파라미터 그대로 평균 35.2° / 최악 91.8°.

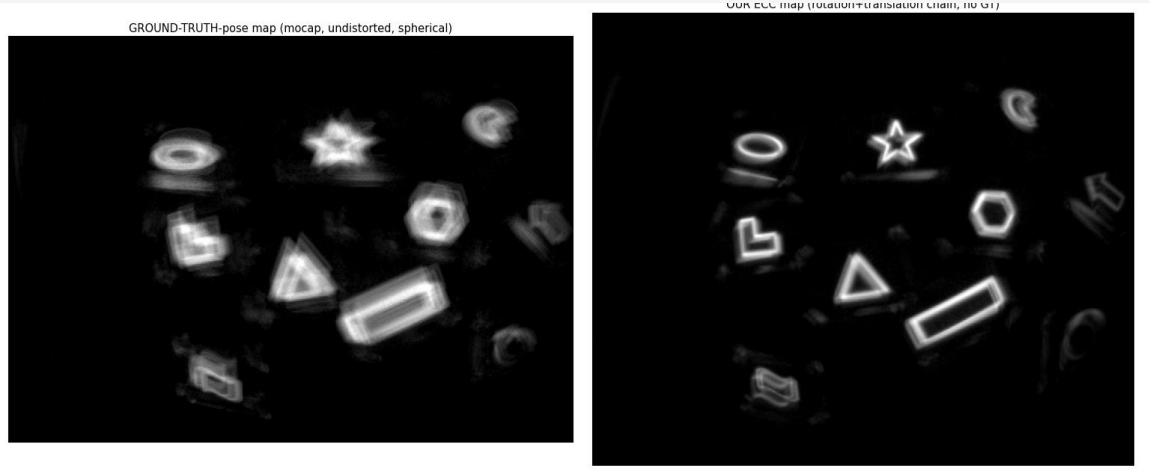
실제 센서 영상으로 만든 월드 지도: 방 구조가 복원된다



실제 DELTA 영상(960× 720) 월드 지도: 첫 결과 · 최종(균형형) · 최종(정제형) · 같은 시점의 실제 프레임(각도 보정 후)

정직한 평가

정답이 없는 실제 영상은 육안과 간접 지표로만 평가

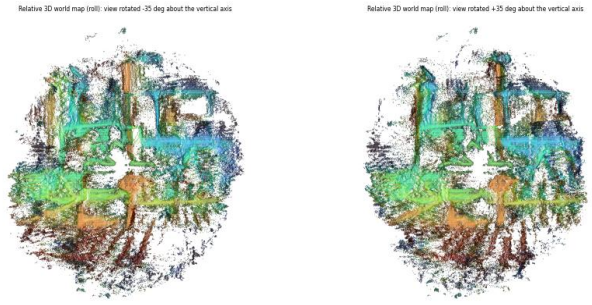


- UZH(모션캡처 정답): 좌 = 정답 자세 맵(시간 어긋남 보정 전이라 번짐), 우 = 우리 ECC 추정 맵
- 소프트웨어(ECC) 추정기: 광축 1초 구간 θ 오차 2.4~3.2° (정답 시간 보정 후)

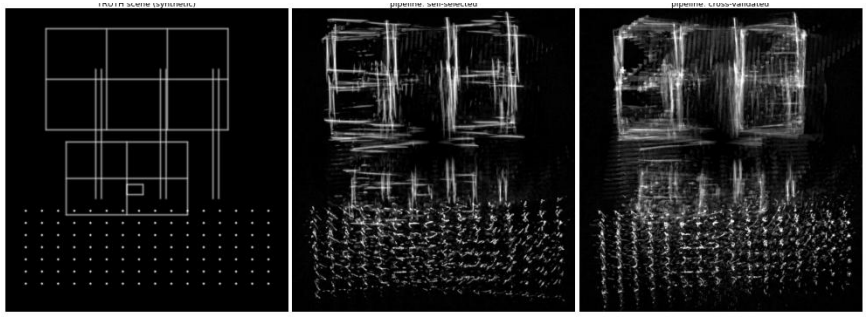
다른 영상과 합성 정답에서도 같은 파이프라인이 동작한다



실제 팬(pan) 영상으로 만든 월드 지도 (한 번에 파노라마로 쌓임)



실제 롤 영상의 상대 3D 점구름

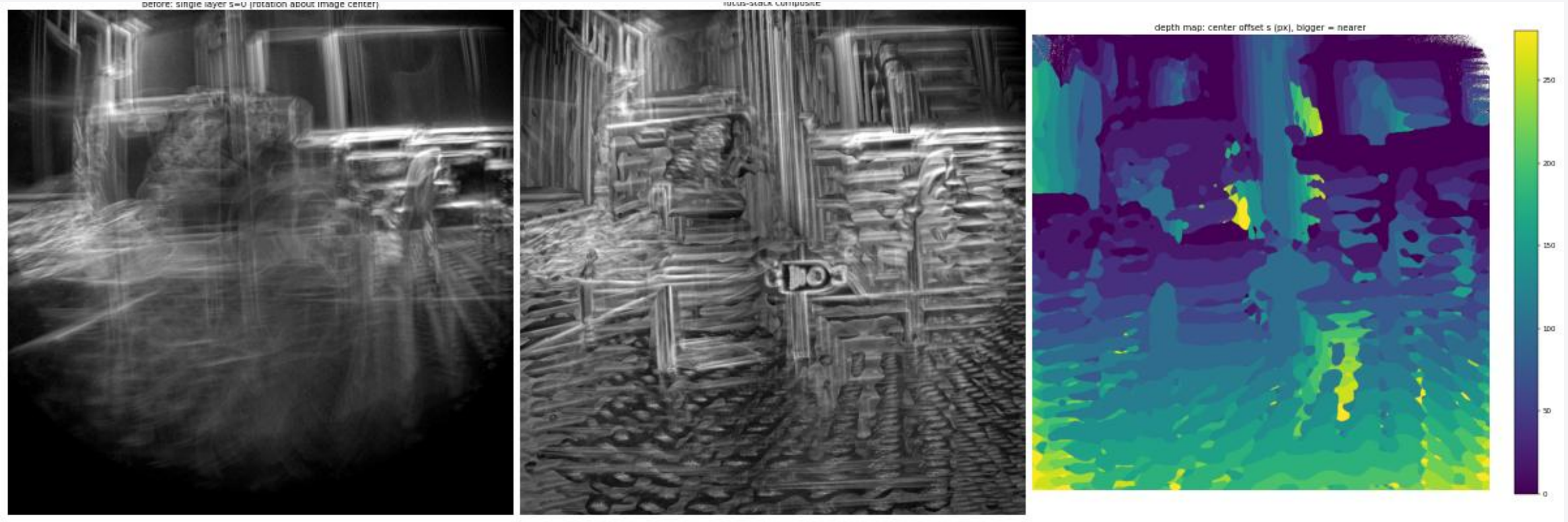


합성 정답 장면 대비 복원 결과

주의

합성 점수는 모델이 정확히 맞는 세계의 값, 실제 영상엔 정답 없음

남은 문제: 회전 중심이 어긋나 가까운 물체는 번진다



회전 중심 하나로 쌓은 지도(좌) → 깊이 층별 지도에서 선명한 층 선택(중) → 추정된 상대 깊이(우)

한계

깊이 층 선택은 소프트웨어 모델이다. RTL에는 회전만 반영했고, 시차 보정은 향후 과제로 남겼다.

생체 모방은 실제 데이터로 이긴 것만 남겼다

생체 모방 아이디어	결과	근거
중심와(fovea) / 이벤트 클러스터링 (1차)	**채택**	열 비트맵: 손실 57.6% → 6.9%(동일 합성 트래픽), 최종 결합판 0.47%(UZH 실측)
눈 깜빡임: 유휴 시 코어 클록 차단	**채택**	항상-켬과 출력·메모리 일치, 유휴 전력 −71%(시험 입력)
행 단위 선택적 활성화 (메모리)	**채택**	전력 −65.5%, 면적 −5%
이득 조절(과부하 솎아내기)	**기각·정정**	합성 장면에서만 이득, 실제 DELTA 프레임에서 이득 없음
추정기 입력 줄이기(ON만/중심 가중/불응기)	**기각**	정확도 1.5배 이상 악화
미세 안구운동(지터)	**기각**	부화소 복원 효과 없음
원심성 복사(VOR): 회전 지령을 θ 사전분포로	**기각**	UZH 평균 23.4°→8.5°이나 RobotEvt 36.1°→34.9°(기준 25.3° 미달), 사전 등록 기준 미통과
최근 쓰기 캐시 (감각 기억)	**보류**	64항목만 전력 −16%/면적 +5.5%, 필수 아님

장단점과 고려 사항: 확장 시 가장 큰 비용은 메모리

장점

- 곱셈기 없는 표 기반 좌표 변환, bit-exact 검증
- 메모리 클록 게이팅(–65.5%)과 blink로 구조적 전력 절감
- 실제 센서·로봇팔 데이터로 검증, 안 되는 것은 기각
- 3차 Q&A로 확인된 물리 셋업(봉-피벗)과 모델이 일치

단점•한계

- 통합 회로의 합성·P&R은 4×4 패치, 64×64 월드, 256각도 (좌표 변환·추정기만 1024×1024로 검증)
- 회전각 추정 정확도가 장면에 크게 의존
- 시차(깊이) 보정은 소프트웨어만, RTL 미반영
- 정답 없는 실제 영상은 간접 지표로만 평가

고려 사항: 목표 크기(1024×1024)로 확장할 때

- 1024×1024 ×2비트 =약 2.1 Mbit. 지금의 FF 메모리 비용(약 12 μm²/비트)로는 약 25 mm² → SRAM 매크로가 필수 (제안 크기 기준 단순 계산, 단순 계산, 합성 결과 아님)
- 교육용 PDK(GPDK045)에는 SRAM 컴파일러가 없어, 메모리 면적·전력은 별도 모델로 추정해야 함
- 쓰기 경로(FIFO·중재·좌표 변환)는 레인 수에 비례해 늘어나지만 메모리에 비하면 작다

요약과 앞으로

한 일

- 회전각으로 이벤트를 월드 좌표로 옮기는 회로를 RTL로 구현
- 표 기반 좌표 변환 (면적 2.5배↓, 전력 4.8배↓)
- 단일 쓰기 포트, 행 클록 게이팅, blink

확인한 것

- 소프트웨어 모델과 비트 단위 일치
- 메모리 전력 − 65.5%
- P&R: DRC·antenna 0건, 클록 게이팅 hold는 ICG로 해결

남은 것

- 1024×1024로의 확장과 SRAM 모델링
- 시차 보정의 RTL 반영
- 실제 영상에서 θ 정확도 개선

주소 자체가 이벤트다 — 필요한 것만 옮기고, 쉬는 동안은 끈다.